

Decoupled Image-to-Video Generation via Latent Flow: An Investigation into Embedding-Conditioned Rendering

Anonymous COMP4471 submission

Paper ID *****

Abstract

001 *Image-to-video generation from a single static image and*
002 *an action label requires simultaneously understanding both*
003 *what is in the scene and how it should evolve over time.*
004 *We investigate a decoupled framework inspired by the Joint-*
005 *Embedding Predictive Architecture (JEPA): a causal Trans-*
006 *former (Flow Model) predicts the trajectory of CLIP em-*
007 *beddings in a compact 512-dimensional semantic space,*
008 *and a conditioned UNet (Renderer) converts each predicted*
009 *embedding back to a pixel frame. We conduct a system-*
010 *atic set of experiments across four Renderer architectures—*
011 *FiLM conditioning, cross-attention to CLIP patch tokens,*
012 *contrastive conditioning loss, and optical flow prediction—*
013 *and find that each variant collapses to a mode where the em-*
014 *bedding condition is effectively ignored and the output is a*
015 *near-identity transformation of the input frame. We present*
016 *a detailed failure analysis identifying the root causes: van-*
017 *ishing embedding motion signal between adjacent frames,*
018 *a training objective fundamentally misaligned with condi-*
019 *tioning sensitivity, and the inherent difficulty of encoding*
020 *spatial motion into a single global feature vector. Despite*
021 *the negative result, our analysis highlights key design chal-*
022 *lenges for embedding-conditioned video synthesis and sug-*
023 *gests concrete alternative approaches.*

024 1. Introduction

025 Image-to-video (I2V) generation aims to produce a tempo-
026 rally coherent video sequence from a single static frame.
027 When conditioned on an action label (e.g., “walking”), the
028 generated video should depict natural motion consistent
029 with that action. This task is relevant to content creation,
030 video editing, and synthetic data augmentation for action
031 recognition systems.

032 Recent large-scale diffusion models such as Stable Video
033 Diffusion and Wan2.2 have demonstrated impressive results
034 by inflating image diffusion architectures into the tempo-
035 ral domain. However, these end-to-end approaches incur

substantial computational costs and remain prone to motion
flickering and identity drift.

Inspired by the JEPA framework, we investigate an al-
ternative: decoupling the problem into a **Flow Model** that
predicts motion in a semantic embedding space and a **Ren-**
derer that converts semantic predictions back to pixels. The
key hypothesis is that motion dynamics are easier to model
in a compact, abstract embedding space than in raw pix-
els. In an ideal scenario, the Flow Model would learn high-
level action dynamics (“a walking person alternates legs”),
and the Renderer would faithfully translate these semantic
blueprints into frames.

A central motivation for using frozen pre-trained en-
coders (CLIP ViT-B/32) was to investigate whether their
rich visual-semantic priors could reduce training costs.
Large-scale video generation models typically require enor-
mous datasets and compute budgets; if a frozen pre-trained
representation already captures meaningful visual structure,
a lightweight decoder might be sufficient to render motion
within that representation space. This is the core hypothe-
sis we set out to test: **can frozen pre-trained visual pri-**
ors serve as an effective backbone for low-cost image-to-
video generation?

We implement this framework using frozen CLIP ViT-
B/32 encoders, a causal Transformer for embedding trajec-
tory prediction, and a UNet-based Renderer with condition-
ing injected via FiLM, cross-attention, or explicit flow pre-
diction heads. We systematically ablate the conditioning
mechanism and loss function across four Renderer variants.

Our key finding is negative but instructive: across all
four variants, the Renderer learns to ignore the embedding
condition and produces near-identical outputs regardless of
input conditioning. The root cause is not any single archi-
tectural choice but a fundamental tension between (1) the
minimal variation in CLIP embeddings between adjacent
video frames, and (2) loss functions that penalize deviation
from a nearly-static ground truth.

We present a detailed failure analysis quantifying the
embedding signal strength, the collapse behavior across
architectures, and the training dynamics. We believe

these findings provide useful guidance for future work on embedding-conditioned generative models.

2. Related Work

2.1. Image-to-Video Generation

Early video generation methods employed recurrent networks or generative adversarial networks. More recently, diffusion-based approaches have become standard. Stable Video Diffusion fine-tunes a pre-trained image diffusion model on video data by adding temporal layers. Wan2.2 follows a similar approach at larger scale. These models generally operate in a VAE latent space, simultaneously modeling spatial structure and temporal dynamics in one pipeline. We experiment with a fundamentally different architecture that attempts to factor these responsibilities across two modules.

2.2. Latent Predictive Architectures

The JEPA framework proposes that predictive learning should operate in representation space rather than input space. V-JEPA applies this principle to self-supervised video learning by predicting masked spatio-temporal features. Our Flow Model follows the same intuition: predict the trajectory of CLIP embeddings rather than raw pixels. Where our work differs is the downstream rendering step—converting predicted embeddings back to pixels—which proves to be the critical bottleneck.

2.3. Conditioned Image Generation

Feature-wise Linear Modulation (FiLM) and cross-attention are standard techniques for conditioning image generation on external information. FiLM applies channel-wise affine transformations $\gamma(c) \odot x + \beta(c)$ predicted from a conditioning vector c , and has been widely used in visual question answering, style transfer, and video prediction. Cross-attention, popularized by the Transformer architecture, allows each spatial position in a feature map to attend to a set of conditioning tokens. Our experiments test both mechanisms independently and in combination, and find that neither is sufficient when the conditioning signal (adjacent-frame CLIP embeddings) varies by less than 3% in cosine distance.

2.4. Optical Flow and Warping

Frame interpolation and video prediction often employ explicit optical flow prediction followed by warping. Methods such as RAFT achieve state-of-the-art optical flow estimation through iterative refinement, while simpler approaches use Farneback or TV-L1 flow as a supervision signal. Our final Renderer variant adopts a flow-based approach: a UNet backbone predicts a dense flow field conditioned on CLIP embeddings, and the previous frame is warped via differentiable bilinear sampling. We use Farneback optical

flow as pseudo-supervision and apply total variation regularization to encourage spatial smoothness.

2.5. Action-Conditioned Video Generation

Motion generation conditioned on action labels has been studied in controlled settings such as video game environments and simplified human motion benchmarks. Text-to-video models such as Make-A-Video and Imagen Video use large language model embeddings for open-vocabulary conditioning at scale. Our setting is intermediate: we use closed-set action labels encoded through CLIP text embeddings, providing a clean semantic signal in the same embedding space as the visual features.

3. Data

We use the HMDB51 dataset, containing 6,754 video clips across 51 human action categories such as “walking,” “drinking,” “brushing hair,” and “climbing stairs.” Videos average approximately 3 seconds in duration at 25–30 fps. The dataset is split into training (70%) and test (30%) sets following the first official split.

3.1. Preprocessing

For each video, we sample 17 consecutive frames ($n = 16$ prediction steps) at a stride of 2, and resize all frames to 224×224 pixels. CLIP embeddings (CLS token + 49 patch tokens) are pre-extracted for the entire dataset (242MB) to avoid redundant forward passes during Flow Model training. The Renderer samples random frame pairs (I_{t-1}, I_t) from each video with a gap of 1 frame.

3.2. Embedding Analysis

A key diagnostic we perform early is measuring the variation in CLIP embeddings across frames. Table 1 summarizes pairwise cosine similarities.

Table 1. CLIP ViT-B/32 embedding cosine similarity. Same-video frames are highly similar ($1 - \cos = 0.024$ for adjacent frames), limiting the information available for frame-to-frame motion prediction.

Comparison	Mean cos
Same video, distant frames ($\Delta t \approx 60$)	0.862
Same video, adjacent frames ($\Delta t = 1$)	0.976
Different videos, same action class	0.664
Different action classes	0.564

The critical observation is that adjacent-frame embeddings differ by only $1 - \cos \approx 0.024$ on average. This means the embedding trajectory that the Flow Model must predict, and that the Renderer must use as a conditioning signal,

contains very little per-frame variation. Any learning algorithm must amplify this tiny signal by orders of magnitude to produce meaningful pixel-space changes.

4. Methods

Our framework consists of three frozen encoders and two trainable modules. The pipeline is as follows: (1) a frozen CLIP ViT encodes the input image I_0 to E_0 and extracts spatial patch tokens; (2) a frozen CLIP text encoder produces the label embedding L ; (3) the Flow Model—a causal Transformer—predicts the embedding trajectory $(\hat{E}_1, \dots, \hat{E}_n)$ from (L, E_0) using shifted teacher forcing; (4) the Renderer—a UNet with FiLM and cross-attention conditioning—produces each frame I_t from I_{t-1} , conditioned on $\hat{E}_t$, L , and patch tokens. The two models are trained separately.

4.1. Encoders

Image Encoder. Frozen CLIP ViT-B/32. We extract both the 512-dim CLS embedding E_t and the 49 spatial patch tokens (7×7 grid, 512-dim each) from the final vision transformer layer.

Label Encoder. Frozen CLIP text encoder. Action labels are converted from directory names to natural text (e.g., “brush hair”) and encoded to 512-dim semantic embeddings L .

4.2. Flow Model

We initially attempted a continuous flow matching formulation: learning a velocity field $v_\theta(E_t, t, L)$ that transports E_0 to E_n through the embedding space. This approach produced unstable trajectories and poor convergence, which motivated our switch to a discrete autoregressive formulation.

Our final Flow Model is a 4-layer causal Transformer encoder (12.9M parameters) with 8 attention heads and feedforward dimension 2048. The input sequence is $[L, E_0, E_1, \dots, E_{n-1}]$, and the model is trained with shifted teacher forcing: position t predicts $\hat{E}_t$ from $(L, E_0, \dots, E_{t-1})$.

We compare two loss functions:

- **MSE:** $\mathcal{L} = \frac{1}{n} \sum_t \|\hat{E}_t - E_t\|_2^2$
- **Cosine:** $\mathcal{L} = \frac{1}{n} \sum_t (1 - \cos(\hat{E}_t, E_t))$

4.3. Renderer

The Renderer is a UNet (4 down/up blocks, 40–43M parameters depending on variant) that takes I_{t-1} as input and produces an output conditioned on $\hat{E}_t$, L , and patch tokens. We experiment with four variants:

1. **FiLM (baseline):** channel-wise Feature-wise Linear Modulation from a condition vector $[\hat{E}_t; L] \in \mathbb{R}^{1024}$, applied at each residual block. Output: pixel residual ΔI_t . Loss: $L1(\Delta I_t, I_t - I_{t-1})$.

2. **FiLM + Cross-Attention:** FiLM conditioning plus multi-head cross-attention from UNet feature maps to CLIP patch tokens at the bottleneck and first two upsampling blocks. Output: pixel residual ΔI_t . Loss: L1.
3. **FiLM + Cross-Attention + Contrastive:** same as variant 2, plus a contrastive conditioning loss: for a batch, we compute $\mathcal{L}_{\text{cont}} = \|\cos(\Delta I_{\text{real}}, \Delta I_{\text{fake}}) - \cos(E_{\text{real}}, E_{\text{fake}})\|_2^2$, where fake conditions are shuffled embeddings from other videos in the batch. This explicitly penalizes the Renderer for producing identical outputs from different conditions.
4. **Flow Prediction:** same UNet backbone with FiLM + cross-attention, but the output head predicts a dense optical flow field $\in \mathbb{R}^{2 \times 224 \times 224}$. The previous frame is warped via the predicted flow: $I_t = \text{warp}(I_{t-1}, \text{flow})$. Loss: $L1(I_t^{\text{warped}}, I_t^{\text{gt}}) + \lambda_{\text{TV}} \cdot \text{TV}(\text{flow}) + \lambda_{\text{FB}} \cdot L1(\text{flow}, \text{Farneback})$.

4.4. Training Details

All models are trained with AdamW ($lr = 10^{-4}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 10^{-2}) and a cosine annealing learning rate schedule over 100 epochs. We use a batch size of 8 with gradient accumulation over 2 steps (effective batch size 16). Gradient clipping with maximum norm 1.0 is applied. All experiments were conducted on a single NVIDIA RTX 4060 Laptop GPU (8GB VRAM). Complete training of the Flow Model takes approximately 4 hours; each Renderer variant takes approximately 7 hours.

Flow Model training data pipeline. Because passing CUDA models through PyTorch DataLoader workers is not supported on Windows, we pre-extract CLIP embeddings for all 6,754 videos (17 frames each) and save them to disk as a single 242MB file. The Flow Model training loop loads pre-extracted embeddings directly, requiring no CLIP forward passes during training. This eliminates the DataLoader CUDA issue and accelerates training by approximately $3 \times$.

Renderer training data pipeline. The Renderer uses HMDB51FrameDataset, which loads raw frames from disk (via OpenCV) and performs image transformations in the DataLoader workers. CLIP encoding happens in the main training loop where CUDA is available. Farneback optical flow is computed on CPU using OpenCV, adding approximately 30% overhead per batch.

4.5. Implementation Notes

Shifted teacher forcing. An earlier version of the Flow Model used “zero-placeholder” inputs where the model received $[L, E_0, \mathbf{0}, \dots, \mathbf{0}]$ and was expected to predict all future embeddings in a single forward pass. This created a train-inference mismatch: at inference time, the model sees its own predictions rather than zeros. Shifted teacher forcing, where the model receives the real embedding sequence

and predicts one step ahead at each position, aligns training and inference exactly. We found this improved embedding trajectory quality significantly (from 2.0% to 82.5% of GT motion, combined with the cosine loss change).

Multi-scale flow prediction. In the flow prediction Renderer variant, we considered predicting flow at multiple decoder resolutions and combining them via coarse-to-fine refinement (similar to RAFT). However, initial experiments showed that even a single-scale flow head struggled to produce consistent non-zero outputs, so we deferred multi-scale investigation to future work.

5. Experiments

5.1. Evaluation Protocol

- We evaluate using the following metrics:
- **Embedding Trajectory MSE:** MSE between predicted and ground-truth CLIP embedding sequences.
 - **Inter-frame 1-cos Distance:** $1 - \cos(\hat{E}_t, \hat{E}_{t+1})$, measuring how much “motion” the predicted embeddings contain.
 - **Conditioning Sensitivity:** $\frac{\| \text{Renderer}(I, E_t) - \text{Renderer}(I, E_0) \|_1}{\| \text{Renderer}(I, E_0) \|_1 + \epsilon}$, measuring whether changing the embedding changes the Renderer output.

5.2. Flow Model Results

Table 2 summarizes the Flow Model results.

Table 2. Flow Model performance with MSE vs. cosine similarity loss. Cosine loss prevents embedding collapse while maintaining prediction accuracy.

Metric	MSE Loss	Cosine Loss
Train loss (final)	0.0116	0.0264
Inter-frame 1-cos (GT)	0.0290	0.0290
Inter-frame 1-cos (pred)	0.0006	0.0239
% of GT motion preserved	2.0%	82.5%

The MSE-trained Flow Model achieves low MSE (0.0116) but collapses to predicting nearly identical embeddings at every timestep (1-cos $\approx$ 0.0006). Cosine loss resolves this: predicted embeddings retain 82.5% of the ground-truth motion variation while maintaining comparable embedding quality (MSE between predicted and GT embeddings actually improves in terms of cosine distance).

5.3. Renderer Results

Table 3 presents the core negative result: across all four architectures, the Renderer learns to ignore the embedding condition.

For all variants, the conditioning sensitivity ratio is approximately 1.0, meaning the magnitude of the Renderer’s output does not meaningfully change when the conditioning

Table 3. Renderer conditioning sensitivity across four architecture variants. A ratio of ~ 1.0 indicates the model produces identical outputs regardless of the embedding condition.

Renderer Variant	Sensitivity Ratio
1. FiLM + L1	1.03x
2. FiLM + Cross-Attn + L1	0.98x
3. FiLM + Cross-Attn + L1 + Contrastive	0.98x
4. FiLM + Cross-Attn + Flow Pred + Rec + TV + FB	1.01x

embedding changes. In variants 1–3 (residual prediction), the absolute output magnitude converges to ≈ 0.005 —essentially near-zero. In variant 4 (flow prediction), the predicted flow field converges back to zero after initial non-zero activity from random initialization.

5.4. Diagnostic Experiments

To isolate the failure mode, we conduct several diagnostic tests:

Embedding distance test. We feed the Renderer embeddings from frames at varying temporal distances ($[t, t + 1, t + 4, t + 8, t + 16]$) and measure the output magnitude. Even with frame $t + 16$ (1-cos = 0.31, a meaningful change), the output magnitude remains unchanged from the $t + 1$ case (1-cos = 0.03). This confirms that the model is not merely limited by the weak signal in adjacent frames—it has learned to ignore the embedding entirely.

Farneback flow as pseudo-target. We compute Farneback optical flow between adjacent frames (mean magnitude 0.39 px, max 2.84 px) and use it as a supervised target. The flow head initially produces non-zero flow but converges back to zero as the reconstruction and smoothness losses dominate.

CLIP feature discrimination. We verify that CLIP embeddings *do* contain frame-to-frame information: the cosine similarity gap between same-video adjacent frames (0.976) and different videos of the same class (0.664) is 0.312—a substantial difference. The information exists, but translating it from a 512-dim global vector into spatially meaningful pixel changes proves fundamentally difficult with our architectures.

6. Discussion: Why Did It Fail?

We identify the following contributing factors, ordered from most fundamental to most specific.

6.1. Primary Cause: Frozen CLIP Embeddings Are Unsuitable for This Task

We believe this is the root cause from which all other issues derive. Several lines of evidence point specifically to the frozen pre-trained ViT embeddings—rather than the Trans-

335	former architecture or the decoupled pipeline design—as	identity transformation: $\Delta I_t \approx 0$ and flow ≈ 0 . When the	387
336	the bottleneck.	conditioning signal carries almost no usable information (as	388
337	First, purely autoregressive Transformer-based video	argued above), identity mapping becomes the rational opti-	389
338	generation has been demonstrated successfully in prior	mum.	390
339	work (e.g., VideoGPT, TATS, and more recent video tok-		
340	enizers combined with autoregressive transformers). These	6.3. Contributing Factor: FiLM Cannot Express	391
341	models generate videos token-by-token or frame-by-frame,	Spatial Guidance	392
342	proving that the autoregressive Transformer paradigm is not	Feature-wise Linear Modulation applies channel-wise scal-	393
343	inherently flawed for video generation. The key difference	ing and shifting uniform across all spatial positions. Even if	394
344	is that these models operate on <i>learned</i> discrete tokens from	the embedding <i>did</i> contain spatial information, FiLM lacks	395
345	a VQ-VAE trained jointly for reconstruction, whereas our	the mechanism to route it to the correct spatial location.	396
346	Flow Model operates on <i>frozen</i> CLIP embeddings that were	Cross-attention to CLIP patch tokens partially addresses	397
347	never optimized for the rendering task.	this routing problem, but the patch tokens themselves still	398
348	Second, CLIP ViT-B/32 is trained with a contrastive	lack dense correspondence information due to the primary	399
349	image-text matching objective: given an image and a text	cause above.	400
350	description, maximize the cosine similarity of their embed-		
351	dings. This objective encourages the model to extract <i>se-</i>	6.4. Limitations of Our Study	401
352	<i>mantic content</i> —“what is in this image”—and compresses	We acknowledge several limitations: (1) we only explored	402
353	it into a compact 512-dimensional vector. What CLIP’s	CLIP ViT-B/32 as the embedding backbone—larger ViT	403
354	training objective does <i>not</i> require is preserving the kind	variants or dense-prediction models like DINOv2 may have	404
355	of information needed for frame-to-frame rendering:	different properties; (2) we did not attempt joint end-to-end	405
356	• Spatial configuration: The precise 2D coordinates of	training of the Flow Model and Renderer, which could al-	406
357	body parts, objects, and their relative positions.	low the Flow Model to learn embeddings specifically opti-	407
358	• Pose and articulation: The joint angles of limbs, the ori-	mized for rendering; (3) we used a relatively small dataset	408
359	entation of a head, the bend of a knee.	(6.7k videos), and the collapse behavior may differ at larger	409
360	• Dense correspondence: Which pixel in frame t corre-	scale where more diverse motion patterns provide a stronger	410
361	sponds to which pixel in frame $t + 1$.	learning signal; (4) HMDB51 videos are constrained to spe-	411
362	A CLIP embedding can tell you “this is a person walk-	cific human action categories, and the learned dynamics	412
363	ing” with high confidence, but it cannot tell you “the right	may not generalize to more open-ended video generation.	413
364	leg is currently forward at a 30-degree angle, positioned at		
365	pixel coordinates (120, 180).” The latter is precisely the in-	6.5. What We Learned	414
366	formation a Renderer needs to predict what the next frame	Despite the negative result, this investigation yielded several	415
367	should look like.	lessons that may benefit future work in this area:	416
368	This explains the asymmetry in our results: the Flow	Embedding choice is critical. The choice of CLIP ViT-	417
369	Model succeeded because predicting an embedding trajec-	B/32 as our embedding backbone was motivated by its joint	418
370	tory only requires matching the statistical distribution of	vision-language space and compact dimensionality. How-	419
371	CLIP embeddings—a task CLIP embeddings contain suf-	ever, CLIP embeddings are optimized for semantic dis-	420
372	ficient information for. The Renderer failed because de-	crimination, not spatial correspondence. For a decoupled	421
373	coding an embedding back to a specific spatial configura-	I2V pipeline to succeed, the embedding space must encode	422
374	tion of pixels requires information that was discarded dur-	both <i>what</i> is in the scene and <i>where</i> things are spatially—	423
375	ing CLIP’s encoding process.	properties that CLIP’s contrastive training objective does	424
376	Empirically, we confirmed this information bottleneck:	not guarantee.	425
377	adjacent frames of the same video have CLIP embeddings	Loss functions define the optimization landscape. We	426
378	with cosine similarity ≈ 0.976 ($1 - \cos \approx 0.024$). This	observed that all tested pixel-space loss functions (L1, L1	427
379	tiny difference must encode the entire motion transforma-	+ TV, L1 + Farneback) contain a near-identity trivial so-	428
380	tion from one frame to the next—a compression ratio far	lution ($\Delta I \approx 0$, flow ≈ 0). This is not specific to our	429
381	exceeding what any decoder architecture can plausibly in-	architecture—any model trained with a pixel reconstruction	430
382	vert.	loss on adjacent-frame pairs faces the same challenge. Loss	431
383	6.2. Contributing Factor: Loss Function Misalign-	functions that operate in a learned feature space (perceptual	432
384	ment	loss, adversarial loss) may provide a stronger signal for mo-	433
385	All our loss functions (L1 on residuals, L1 on reconstruc-	tion learning.	434
386	tion after warping) have an optimal trivial solution near the	The JEPA philosophy needs a rendering pathway.	435
		JEPA’s core insight—predict in representation space, not	436

437	input space—proved effective for our Flow Model (82.5%	cross-attention, contrastive conditioning, and optical flow	488
438	of GT motion). But the companion challenge of <i>decod-</i>	prediction.	489
439	<i>ing</i> those predictions back to the pixel domain is equally		
440	hard and received comparatively little attention in our im-	Our analysis suggests the primary cause is a fundamen-	490
441	plementation. The rendering problem deserves its own care-	tal information bottleneck: CLIP ViT-B/32 embeddings are	491
442	ful treatment rather than being treated as an architectural	trained for semantic image-text alignment and do not pre-	492
443	afterthought.	serve the spatial configuration, pose, and correspondence	493
		information necessary for pixel-level frame rendering. Ad-	494
444	6.6. Future Work	ja-cent frames differ by less than 3% in CLIP embedding	495
445	Several promising directions emerge from this analysis.	cosine distance, an information compression ratio that is	496
446	Dense spatial embeddings. Replacing CLIP with	implausible for any decoder architecture to invert. All	497
447	a model trained for dense visual correspondence—such	other observed problems—loss function collapse to identity,	498
448	as DINOv2, Stable Diffusion’s VAE latents, or a self-	FiLM’s inability to express spatial motion—can be under-	499
449	supervised optical flow network—could provide embed-	stood as consequences of this root cause.	500
450	dings that naturally encode spatial configuration. A tra-		
451	jectory in DINOv2 patch feature space would carry richer	Returning to our original research question—whether	501
452	spatial information than a trajectory in CLIP CLS space.	frozen pre-trained visual priors can serve as an effective	502
453	Joint end-to-end training. Training the Flow Model	backbone for low-cost I2V generation—our results suggest	503
454	and Renderer jointly, with gradients flowing from the ren-	a nuanced answer. The priors <i>do</i> carry useful semantic	504
455	dering loss back through the Flow Model, would allow the	information: our Flow Model successfully learned action-	505
456	embedding space to adapt to the needs of the Renderer.	conditioned embedding trajectories with 82.5% of ground-	506
457	The Flow Model would no longer be constrained to match	truth motion variation, and the embedding space cleanly	507
458	CLIP’s embedding structure and could learn features specifi-	separates different action classes (inter-class 1-cos $\approx$ 0.44	508
459	cally optimized for frame rendering.	vs. intra-class $\approx$ 0.34). However, semantic priors alone	509
460	Iterative refinement. Rather than predicting the full	are insufficient for the <i>rendering</i> stage, which requires fine-	510
461	frame transformation in a single forward pass, a multi-step	grained spatial information that CLIP’s training objective	511
462	refinement process (similar to diffusion models or recurrent	discards. The lesson is not that pre-trained priors are use-	512
463	refinement) could gradually build up the output from an ini-	less for video generation, but that the choice of pre-trained	513
464	tial near-identity estimate. This would allow the model to	representation must be matched to <i>both</i> stages of a decou-	514
465	learn motion in stages—coarse structure first, then fine de-	pled pipeline—the prediction stage needs semantic struc-	515
466	tails.	ture, while the rendering stage needs spatial correspon-	516
467	Adversarial and perceptual losses. GAN-based losses	dence.	517
468	or CLIP-based perceptual losses directly penalize lack of	Future directions suggested by this analysis include:	518
469	motion and unrealistic static output, potentially breaking	(1) replacing CLIP with a dense visual representation	519
470	the near-identity local minimum that trapped all of our L1-	trained for spatial correspondence, such as DINOv2 patch	520
471	trained models.	features, Stable Diffusion VAE latents, or self-supervised	521
472	Explicit motion field supervision. Using a pre-trained	optical flow features; (2) training the Flow Model and Ren-	522
473	optical flow network (e.g., RAFT) instead of Farneback as	derer jointly end-to-end, so the embedding space can adapt	523
474	a supervision target could provide a cleaner learning signal.	to the specific needs of rendering rather than being con-	524
475	Combined with a rendering loss, this could guide the model	strained by a frozen pre-trained encoder; (3) adopting losses	525
476	toward physically plausible motion while maintaining visu-	that explicitly penalize lack of motion, such as adversar-	526
477	al fidelity.	ial or perceptual losses, to break the near-identity local	527
		minimum; (4) incorporating explicit motion representations	528
478	7. Conclusion	such as optical flow or keypoint trajectories as intermediate	529
479	We set out to build a decoupled image-to-video genera-	supervision targets rather than relying solely on the embed-	530
480	tion system in which a causal Transformer predicts action-	ding to carry all motion information.	531
481	conditioned embedding trajectories and a conditioned UNet		
482	renders those trajectories to pixels. The Flow Model com-	References	532
483	ponent was partially successful: with cosine similarity	[1] A. Blattmann, T. Dockhorn, S. Kulal, et al. “Stable Video	533
484	loss it learned to predict embedding trajectories preserving	Diffusion: Scaling Latent Video Diffusion Models to Large	534
485	82.5% of ground-truth motion variation, while MSE loss	Datasets.” <i>arXiv:2311.15127</i> , 2023.	535
486	led to complete collapse. However, the Renderer failed	[2] Alibaba. “Wan2.2: Large-Scale Image-to-Video Genera-	536
487	across all four conditioning architectures tested: FiLM,	tion.” 2025–2026.	537
		[3] Y. LeCun. “A Path Towards Autonomous Machine Intelli-	538
		gence.” <i>OpenReview</i> , 2022.	539

[4] A. Bardes, Q. Garrido, J. Ponce, et al. “Revisiting Feature Prediction for Learning Visual Representations from Video.” *arXiv:2404.08471*, 2024.

[5] A. Radford, J. W. Kim, C. Hallacy, et al. “Learning Transferable Visual Models From Natural Language Supervision.” *ICML*, 2021.

[6] H. Kuehne, H. Jhuang, E. Garrote, et al. “HMDB: A Large Video Database for Human Motion Recognition.” *ICCV*, 2011.

[7] E. Perez, F. Strub, H. de Vries, et al. “FiLM: Visual Reasoning with a General Conditioning Layer.” *AAAI*, 2018.

[8] R. Zhang, P. Isola, A. A. Efros, et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” *CVPR*, 2018.

[9] A. Vaswani, N. Shazeer, N. Parmar, et al. “Attention Is All You Need.” *NeurIPS*, 2017.

[10] P. Isola, J.-Y. Zhu, T. Zhou, et al. “Image-to-Image Translation with Conditional Adversarial Networks.” *CVPR*, 2017.

[11] O. Ronneberger, P. Fischer, T. Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” *MICCAI*, 2015.

[12] J. Oh, X. Guo, H. Lee, et al. “Action-Conditional Video Prediction using Deep Networks in Atari Games.” *NIPS*, 2015.

[13] N. Srivastava, E. Mansimov, R. Salakhutdinov. “Unsupervised Learning of Video Representations using LSTMs.” *ICML*, 2015.

[14] C. Vondrick, H. Pirsivash, A. Torralba. “Generating Videos with Scene Dynamics.” *NIPS*, 2016.

[15] U. Singer, A. Polyak, T. Hayes, et al. “Make-A-Video: Text-to-Video Generation without Text-Video Data.” *arXiv:2209.14792*, 2022.

[16] J. Ho, W. Chan, C. Saharia, et al. “Imagen Video: High Definition Video Generation with Diffusion Models.” *arXiv:2210.02303*, 2022.

[17] M. Monfort, A. Andonian, B. Zhou, et al. “Moments in Time Dataset: One Million Videos for Event Understanding.” *TPAMI*, 2019.